

DETECÇÃO DE PLACAS VEICULARES COM MOBILENET V1 & GRID DETECTOR

Trabalho Final de Processamento Digital de Imagens

Arquitetura: MobileNet V1 (Alpha 0.5)

Algoritmo: Spatial Grid Detection (YOLO-style)

Carlos Vinícius

Victor Guedes

Alisson Jaime

Kaique Ferreira

| CONTEXTO E OBJETIVOS

O Objetivo

Desenvolver um sistema capaz de detectar automaticamente **placas veiculares** em imagens, mantendo boa precisão mesmo diante de diferentes posições, escalas e condições de iluminação e otimizado para **embarcar**.

A Solução Proposta

Projetar um modelo leve de detecção utilizando **MobileNetV1** e um detector baseado em grade (Grid Detector), permitindo sua execução em **dispositivos embarcados** com recursos computacionais limitados.

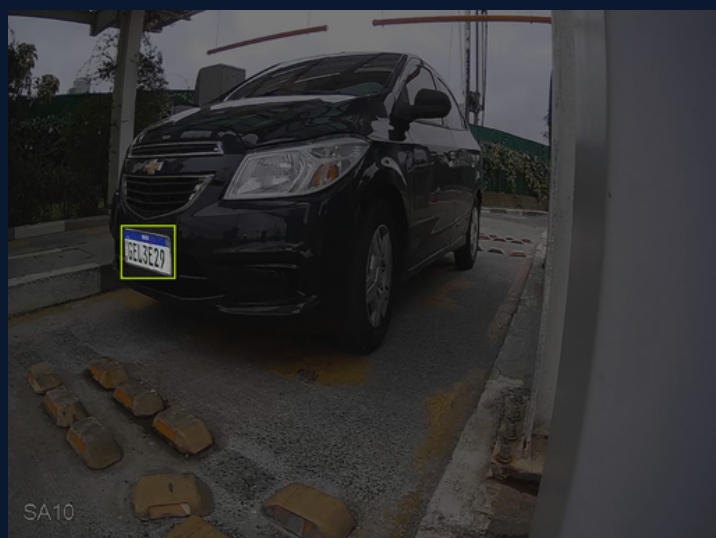
Escopo do projeto

- ✓ Treinamento de um detector baseado em **MobileNetV1**;
- ✓ Localização da placa por meio de um **Grid Detector**;
- ✓ Segmentação dos caracteres com técnicas de **PDI**;
- ✓ Conversão do modelo para **TensorFlow Lite INT8**;
- ✓ Exportação para **Sistemas Embarcados (ESP32-S3)**.

| Base de Dados

Bases Utilizadas

Foram utilizadas **duas bases de dados** contendo imagens de placas veiculares anotadas no formato **YOLO**. As bases foram **combinadas** para aumentar a diversidade de exemplos e melhorar a capacidade de generalização do modelo.



Base	Treino	Validação
Base do professor	350	60
Brazil Plates Detector	200	12
Mix ponderado	972	72
Imagens negativas	12	4

| Etapas De Construção Do Sistema

TREINAMENTO

- Bases de Dados
- Aumento de Dados
- Treinamento
- Fine-Tuning

PRÉ-PROCESSAMENTO

- Redimensionamento
- Normalização
- Conversão RGB/gray3

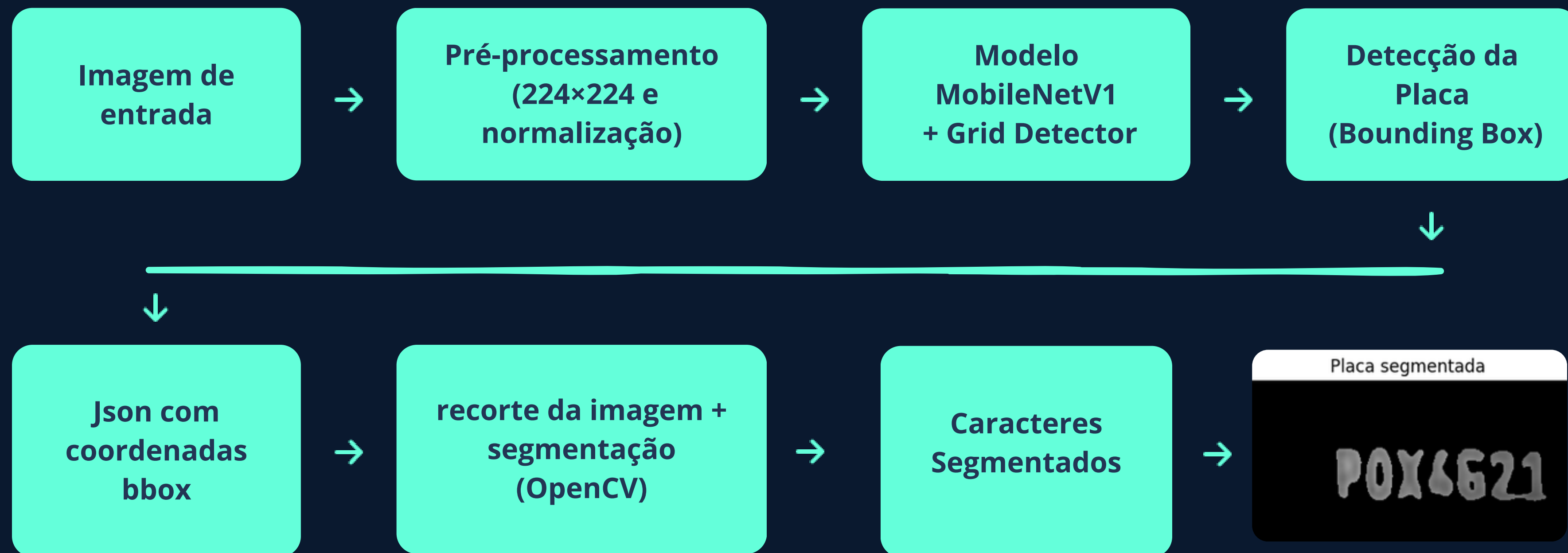
DETECÇÃO

- MobileNetV1
- Grid Detector
- Bounding Box
- Coordenadas

PÓS-PROCESSAMENTO

- Recorte da ROI
- CLAHE/BlackHat/Otsu
- Segmentação
- Imagem Final

| Fluxo Geral do Sistema



| Pré-processamento das Imagens

Padronização das imagens antes do treinamento da rede neural.

Redimensionamento: 224×224

Normalização: -1 a 1

RGB/gray3: compatível com MobileNetV1

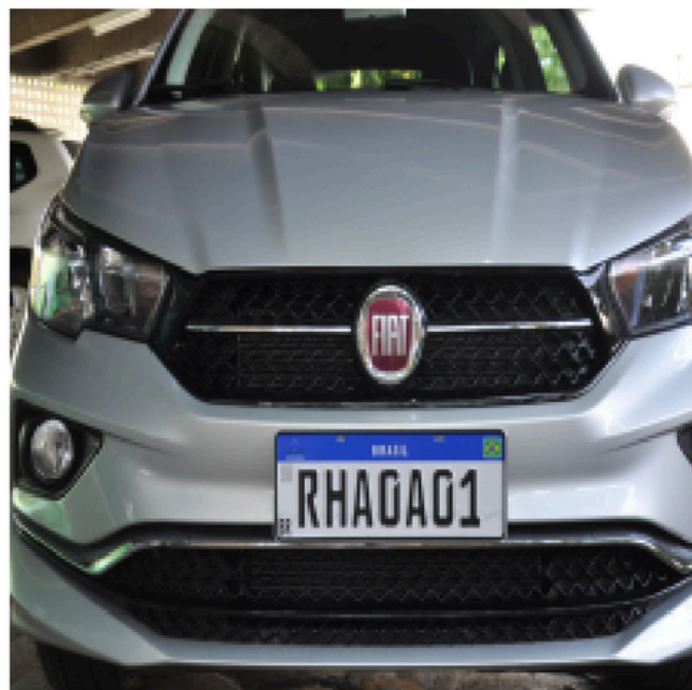
Aumento de dados: mais variação

$$x_{\text{norm}} = \frac{x}{127,5} - 1$$

Original



224×224



Normalizada
[-1, 1]



gray3



Augmentada



| MobileNetV1: Arquitetura Escolhida



Por que MobileNetV1?

- Arquitetura leve e eficiente;
- Baixo custo computacional;
- Adequada para sistemas embarcados.



Configuração Utilizada

- Entrada: $224 \times 224 \times 3$
- $\alpha = 0,5$
- Pesos iniciais: ImageNet



Papel no Projeto

Extrair características da imagem para que o Grid Detector localize a placa veicular.

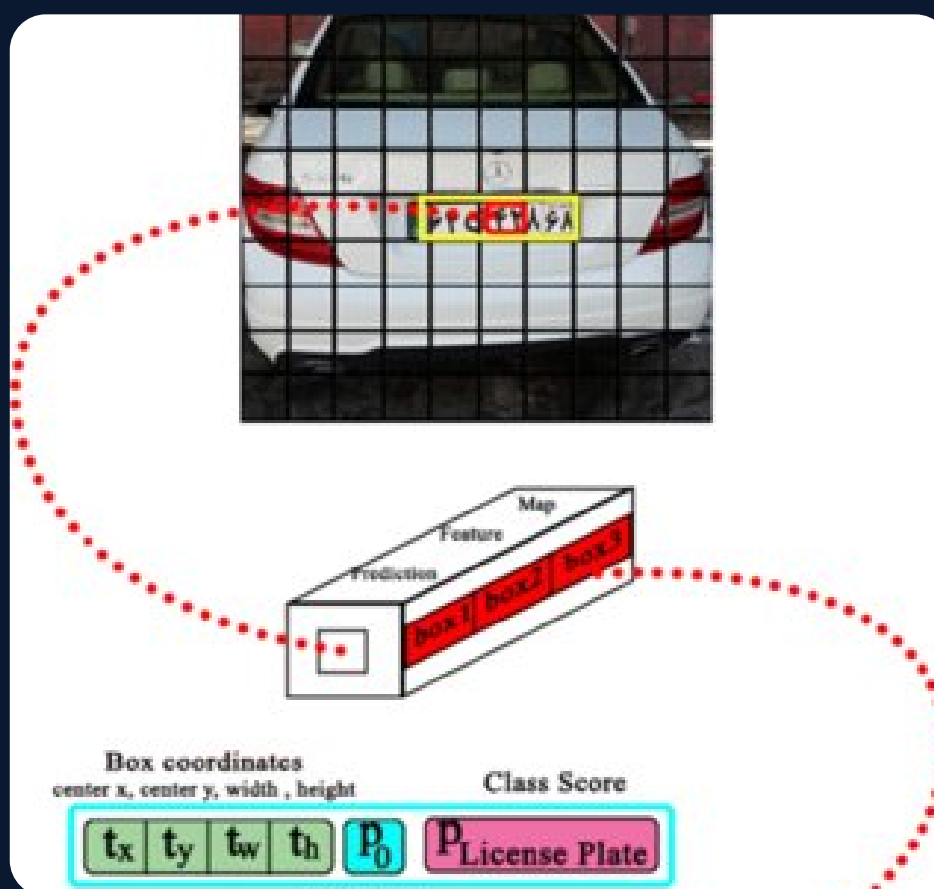
A MobileNetV1 realiza a extração de características da imagem de entrada, produzindo mapas de características que são utilizados pelo detector por grade para localizar a placa veicular de forma eficiente e com baixo custo computacional.

| Detector por Grade (Grid Detector)

Funcionamento

A imagem é dividida em uma grade 7×7 , onde cada célula é responsável por prever a presença de uma placa em sua região.

Saída da Rede



Resultado

(confiança, cx, cy, w, h)
Esse resultado é usado para fazer a segmentação no local correto, onde é cortada a imagem no local da bbox e em seguida aplicadas técnicas de PDI para obtermos o melhor resultado visual dos caracteres das placas

| FLUXO DE TREINAMENTO E OTIMIZAÇÃO

Fase 1: Estratégia do Treinamento

- Transfer Learning
- Congelamento inicial do backbone
- Fine-tuning da rede

Fase 2: Configurações

- Batch Size: 8
- 12 + 50 épocas
- Otimizador: Adam
- EarlyStopping e ReduceLROnPlateau

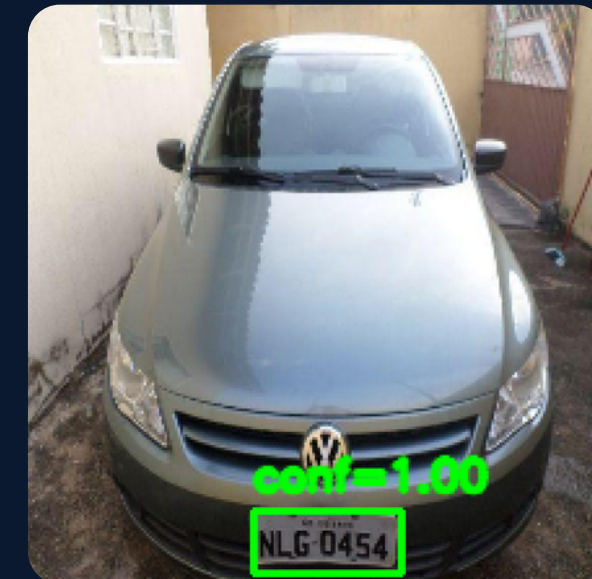
Função de Perda (Custom Loss)

A perda combina **Erro Quadrático (Confiança)** + **Smooth L1 (BBox)** + **GloU Loss** para maximizar a sobreposição real.

| Análises Visuais Realizadas

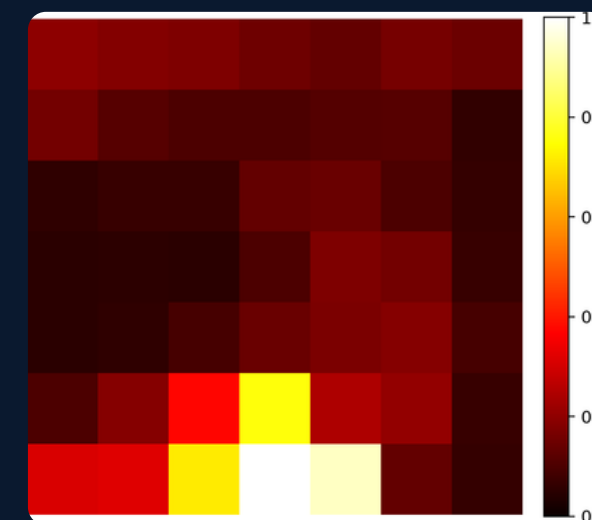
Bounding Box

Localização da
placa na imagem.



Mapa de Confiança

Mostra qual célula
da grade recebeu
maior ativação.



Segmentação

Recorte da ROI e
extração visual
dos caracteres.



| Pós-processamento da Detecção

Entrada do algoritmo

1. Recorte da **ROI** via BBox prevista.
2. Filtro Bilateral e CLAHE (Contraste).
3. **BlackHat**: Realce de caracteres escuros.
4. **Otsu Threshold**: Binarização inteligente.
5. Limpeza Morfológica e Filtro de Componentes.

Técnicas Aplicadas

CLAHE;
BlackHat;
Limiarização (Otsu);
Filtragem morfológica.

Resultado Esperado



| DEPLOY EM SISTEMAS EMBARCADOS

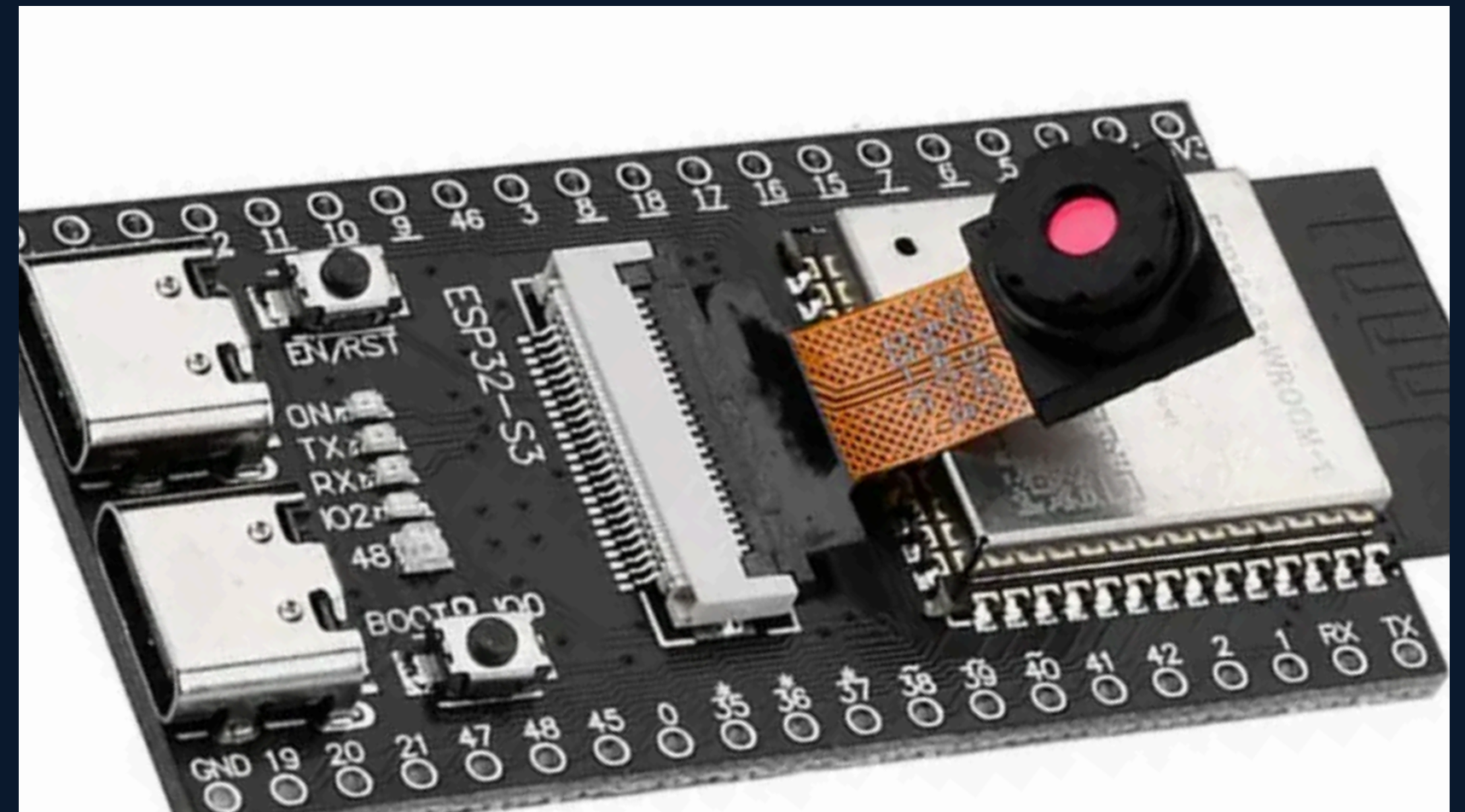
Otimização TFLite

Conversão do modelo Keras para formato **TensorFlow Lite**, utilizando quantização de pesos para reduzir o consumo de memória e energia.

Target:ESP32-S3 CAM OV5640

Conversão para **Integer Quantization (INT8)**.

Geração de arquivo .h (header C++).



| ANÁLISE DE RESULTADOS EM CAMPO

Precisão

Métrica	Resultado
IoU Médio	0,731
Recall	0,922
F1-Score	0,944

Desempenho

Plataforma	Inferência
PC	4-7 ms
ESP32-S3	117 s

| A quantização INT8 reduziu significativamente o tamanho do modelo sem perda relevante de precisão.

CONCLUSÃO

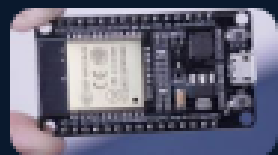
A abordagem de **Grid Detection** com MobileNet V1 provou ser superior à regressão direta, mantendo boa precisão na detecção de placas. A conversão para TFLite INT8 viabilizou o embarcamento inicial na ESP32-S3, embora o tempo de inferência ainda seja a principal limitação prática.



Dúvidas?

Obrigado pela atenção.

| Links dos datasets



<https://wevolver-project-images.s3.us-west-1.amazonaws.com/0.9qiedli837ghamed-taha-NEqR20e6eY4-unsplash-Edited1.jpg>

Source: www.wevolver.com



https://static.vecteezy.com/system/resources/previews/052/985/877/non_2x/computer-vision-icon-artificial-intelligence-concept-vector.jpg

Source: www.vecteezy.com